

Water delivery, with evidence.

JalSetu Lite connects citizens, public water providers, and private tanker operators through transparent offers and verifiable delivery steps.

PRICE VISIBILITY	DELIVERY VERIFICATION	OPERATOR FEEDBACK
------------------	-----------------------	-------------------

90-SECOND PITCH

Finding water during a shortage should not mean calling around, guessing whether a price is fair, and hoping the tanker delivers what was promised.

JalSetu Lite is a transparent coordination platform for citizens, public water providers, and private tanker operators. A citizen selects a location, water type, and tanker size, then compares nearby offers against public water-rate benchmarks. If a private quote exceeds the benchmark, the app flags it before the citizen confirms.

JalSetu also addresses the trust problem after booking. Operators dispatch orders, drivers confirm arrival, and delivery is checked with a one-time code and a meter reading. A short delivery or repeated incorrect codes moves the order into a dispute state. Completed orders and feedback contribute to a clearer record of operator performance.

We are presenting a working prototype and a foundation for a local pilot, not a deployed water network. Our next step is to work with a municipal partner and participating operators, then measure whether transparent quotes and delivery verification improve trust, completion, and price visibility.

A transparent coordination layer for local water logistics.

The product and the demo

JalSetu takes an order from location-based discovery through delivery evidence, then makes the outcome visible to both sides.

The user journey

1	Compare A citizen chooses location, water type, and tanker capacity. Nearby public and private offers are shown with distance, price, and a public-rate warning when applicable.
2	Confirm The citizen confirms an offer and receives a delivery code. The current QR is a demo-only payment request; it does not transfer or verify money.
3	Dispatch and verify The operator dispatches the assigned order. The driver records arrival, enters the code, and submits meter readings. The backend checks that recorded delivery meets the configured 95 percent threshold.
4	Resolve and learn Repeated incorrect codes or a short meter reading can place the order in DISPUTED. After a delivered, disputed, or cancelled result, a citizen can rate or comment; operators have a public feedback summary.

Suggested live demo

SHOW	WHAT JUDGES SHOULD NOTICE
Citizen offers	Compare offers and see the public benchmark flag before confirming.
Operator and driver	Show assigned-order dispatch, delivery code, and meter verification.
Order result	Show the final status and the feedback prompt or the Rate or edit action in account history.
Feedback drawer	Show average, rating distribution, and newest-first operator reviews.

Architecture and technology

The core workflow is a single-origin web application: FastAPI serves the API and the HTML dashboards, while SQLite stores accounts, orders, events, and feedback.

LAYER	TECHNOLOGY	ROLE IN THE PRODUCT
API and business logic	Python 3.12, FastAPI, Uvicorn, Pydantic	REST endpoints, validation, authorization, order state changes
Data	SQLite, WAL mode	Orders, users, sessions, events, feedback; appropriate to the current single-instance prototype
Citizen and staff apps	HTML, CSS, JavaScript	Citizen request and history; operator dispatch; driver arrival and delivery
Maps	Leaflet, OpenStreetMap, Nominatim, Photon fallback	Location selection, map display, address search
Scorecard source	Next.js 16, React 19, TypeScript 5.7, Tailwind CSS 4	Separate scorecard subproject; the main container serves the static frontend
Operations	Docker, Docker Compose, GitHub Actions	Container build, health checks, persistent volume, CI regression and smoke checks

Trust and operational controls

- + Citizen accounts self-register. Operator and driver accounts are provisioned by an administrator; API routes check roles and order ownership.
- + Passwords are stored as salted PBKDF2-HMAC-SHA256 hashes. Session tokens are random bearer tokens stored server-side as hashes and expire after 12 hours by default.
- + Delivery codes are verified with server-side HMAC-SHA256. Three incorrect attempts or a meter shortfall can put an order into dispute.
- + Production configuration requires separate strong OTP and administrator secrets, explicit CORS origins, and HTTPS for non-local origins.
- + Docker uses a non-root account, a read-only application filesystem, a persistent database volume, health checks, and resource limits.

Data model: orders, users, auth_sessions, order_events, and vendor_feedbacks. Operator identity comes from the seeded operator catalog; the current project does not have a separate vendors table or ORM.

Pilot plan and judge Q&A;

The product is testable, but its social and operational impact still needs field evidence. Treat the pilot as the next validation step.

Pilot measures to collect

- + Price visibility: share of requests with multiple offers and the difference between private quotes and the public benchmark.
- + Fulfilment: confirmation-to-arrival time, verified delivery rate, cancellations, and disputes.
- + Trust: citizen feedback response rate, rating distribution, and whether review information changes offer selection.
- + Operations: operator onboarding effort, driver workflow completion, and support cases per order.

Questions judges may ask

Has JalSetu reduced prices or improved delivery? Not yet measured in a field pilot. The prototype exposes benchmarks and records delivery outcomes; those measures must be tested with partners.

Does the QR process payments? No. The QR uses a deliberately invalid demo payee. There is no payment provider callback, settlement confirmation, or escrow.

Can the current database scale to a city-wide, multi-instance service? SQLite is suitable for the current single-instance prototype. A larger deployment should move to PostgreSQL and add production monitoring, backups, and operational support.

Where do the benchmarks and operators come from? Current operator and tariff data are seeded in project configuration. Live municipal feeds and a verified operator onboarding process are future integration work.

Current readiness

Implemented: citizen, operator, and driver journeys; role-based accounts; order history; OTP and meter checks; dispute transitions; feedback submission and operator summaries; SQLite backup and restore utilities.

Verified in local checks: 19 backend tests and 5 frontend tests passed in the latest recorded runs. GitHub Actions also defines backend, frontend, Compose, and container readiness checks.

Not yet proven: deployed municipal integration, live payment settlement, verified regional operator coverage, field impact results, and multi-instance scale.

The ask

Invite a municipal water team and local tanker operators to co-design a bounded pilot, validate the operating rules, and agree on outcome measures before expanding coverage.